TP2 - Foundations of Parallel Computing

Khaoula Mejhoudi

February 5, 2026

1 Exercise 1: Loop Optimizations

1.1 Results with -O0

Factor	Time (ms)	Speedup
U=1	3.58	1.00x
U=2	1.81	1.98x
U=4	1.51	2.37x
U=8	1.68	2.13x
U=16	1.17	3.06x
U=32	1.25	2.86x

1.2 Results with -O2

Factor	Time (ms)	Speedup
U=1	1.47	1.00x
U=2	0.87	1.69x
U=4	0.69	2.13x
U=8	0.75	1.96x
U=16	0.98	1.50x
U=32	1.43	1.03x

1.3 Q3: Best unrolling factor

With -O0: U=16 gives best performance (1.17 ms).

With -O2: U=4 gives best performance (0.69 ms).

1.4 Q4-Q5: Manual unrolling vs compiler optimization

- U=1 with -O0: 3.58 ms
- U=1 with -O2: 1.47 ms (2.4x faster)
- U=4 with -O2: 0.69 ms (best)

Manual unrolling at -O0 can beat -O2 without unrolling. Combining both gives best results.

1.5 Q6: Different data types

Type	Size	U=1 (ms)	U=8 (ms)
double	8 bytes	3.57	1.35
float	4 bytes	3.44	1.39
int	4 bytes	2.96	1.64
short	2 bytes	2.60	1.14

Smaller types are faster because they use less memory bandwidth.

1.6 Q7: Memory bandwidth

Theoretical minimum: $T_{min} = \frac{N \times sizeof(type)}{BW}$

With BW = 25 GB/s: T_{min}(double) = 0.32 ms, but measured = 1.35 ms.

Measured times are higher due to loop overhead, pipeline stalls, and memory latency.

1.7 Q8: Why performance saturates

- Small U (1-4): Improves performance (less loop overhead, more ILP).
- Medium U (4-8): Best performance (matches AVX width and cache line).
- Large U (16-32): Degrades (register spilling, larger code, bandwidth limit).

2 Exercise 2: Instruction Scheduling

2.1 Q1: Execution time comparison

Version	Time (s)
Original -O0	0.342
Original -O2	0.130

Speedup with -O2: 2.6x faster than -O0.

2.2 Q2: Compiler optimizations at -O2

Assembly with -O0 (loop body):

```
.L3:
    movsd  xmm0, [rbp-32]    ; load a
    mulsd  xmm0, [rbp-24]    ; xmm0 = a*b
    movsd  xmm1, [rbp-48]    ; load x
    addsd  xmm0, xmm1        ; xmm0 = a*b + x
    movsd  [rbp-48], xmm0    ; store x
    movsd  xmm0, [rbp-32]    ; load a (again)
    mulsd  xmm0, [rbp-24]    ; xmm0 = a*b (again)
    movsd  xmm1, [rbp-40]    ; load y
    addsd  xmm0, xmm1        ; xmm0 = a*b + y
    movsd  [rbp-40], xmm0    ; store y
    add    [rbp-52], 1       ; i++
```

Assembly with -O2 (loop body):

```
.L2:
    addsd  xmm1, xmm0        ; x += ab
    addsd  xmm1, xmm0        ; y += ab (merged)
    sub    eax, 2            ; i -= 2 (unrolled)
    jne    .L2
```

Main optimizations:

- Constant folding: $a \times b$ computed once before loop
- Loop unrolling: 2 iterations per loop (sub eax, 2)
- Register allocation: values stay in registers (xmm0, xmm1)
- Dead code elimination: x and y merged (same value)
- No memory access inside loop

2.3 Q3: Manual optimization vs -O2

Manually optimized code (precompute $a \times b$, unroll by 2):

Version	Time (s)	Speedup vs Original -O0
Original -O0	0.342	1.00x
Manual -O0	0.306	1.12x
Original -O2	0.130	2.63x

Conclusion: Manual optimization with -O0 (0.306 s) is faster than original -O0 (0.342 s), but still much slower than -O2 (0.130 s). The compiler applies more optimizations than we can easily do by hand (register allocation, merging variables, eliminating all memory access).

3 Exercise 3: Amdahl and Gustafson

3.1 Q1: Code Analysis

Sequential part:

- `add_noise`: each element depends on the previous one ($a[i] = a[i-1] \times 1.0000001$)

Parallelizable parts:

- `init_b`: no dependencies between iterations
- `compute_addition`: no dependencies between iterations
- `reduction`: can be parallelized with reduction pattern

Time complexity: All functions are $O(N)$.

3.2 Q2: Sequential Fraction (Callgrind)

Profiled with: `valgrind --tool=callgrind ./exercise3`

Function	Instructions	Type
add_noise	30.30%	Sequential
init_b	18.18%	Parallel
compute_addition	36.36%	Parallel
reduction	15.15%	Parallel

Sequential fraction: $f_s = 0.303$ (30.3%)
Parallel fraction: $f_p = 1 - f_s = 0.697$ (69.7%)

3.3 Q3: Strong Scaling (Amdahl's Law)

Amdahl's Law: $S(p) = \frac{1}{f_s + \frac{1-f_s}{p}}$

Processors (p)	Speedup S(p)
1	1.00
2	1.53
4	2.10
8	2.56
16	2.89
32	3.08
64	3.19

Maximum theoretical speedup: $S_{max} = \frac{1}{f_s} = \frac{1}{0.303} = 3.30$

Why speedup saturates: As $p \to \infty$, the parallel part takes zero time, but the sequential part still takes the same time. The sequential fraction limits the maximum speedup.

3.4 Q5: Weak Scaling (Gustafson's Law)

Gustafson's Law: $S(p) = f_s + p \times (1 - f_s)$

Processors (p)	Speedup S(p)
1	1.00
2	1.70
4	3.09
8	5.88
16	11.46
32	22.61
64	44.91

Comparison:

- Amdahl: fixed problem size, speedup limited by sequential fraction
- Gustafson: problem size grows with p, speedup scales linearly
- Gustafson is more optimistic because larger problems have relatively smaller sequential overhead

4 Exercise 4: Matrix Multiplication

4.1 Code Analysis

Sequential part:

- `generate_noise`: $O(N)$ - each element depends on previous

Parallelizable parts:

- `init_matrix`: $O(N^2)$ - no dependencies
- `matmul`: $O(N^3)$ - highly parallelizable (dominates runtime)

4.2 Sequential Fraction (Callgrind)

Function	Instructions	Type
generate_noise	0.0003%	Sequential
init_matrix (2x)	0.60%	Parallel
matmul	99.38%	Parallel

Sequential fraction: $f_s \approx 0.000003$ (0.0003%)
This is extremely small because `matmul` is $O(N^3)$ while `generate_noise` is only $O(N)$.

4.3 Strong Scaling (Amdahl's Law)

With $f_s = 0.000003$:

Processors (p)	Speedup S(p)
1	1.00
2	2.00
4	4.00
8	8.00
16	16.00
32	32.00
64	64.00

Maximum theoretical speedup: $S_{max} = \frac{1}{f_s} \approx 333,333$
The speedup is nearly linear because the sequential fraction is negligible.

4.4 Weak Scaling (Gustafson's Law)

Processors (p)	Speedup S(p)
1	1.00
2	2.00
4	4.00
8	8.00
16	16.00
32	32.00
64	64.00

Both Amdahl and Gustafson give nearly identical (linear) results because $f_s \approx 0$.

4.5 Comparison with Exercise 3

	Exercise 3		Exercise 4	
f_s	30.3%		0.0003%	
p	Amdahl	Gustafson	Amdahl	Gustafson
8	2.56	5.88	8.00	8.00
64	3.19	44.91	64.00	64.00
S_{max}	3.30	∞	333,333	∞

4.6 Speedup Plots

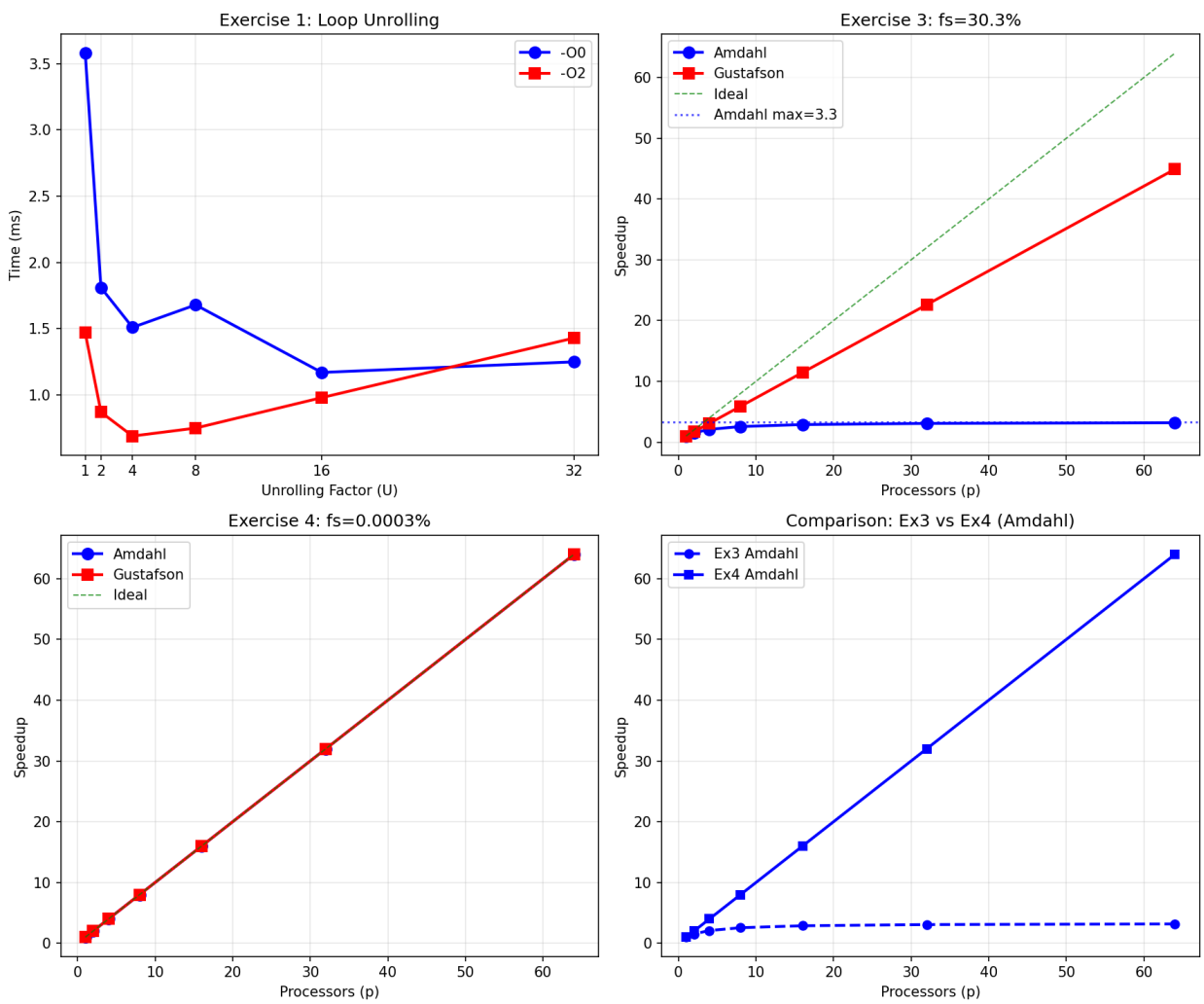

Figure 1: Speedup curves: (top-left) Exercise 1 loop unrolling, (top-right) Exercise 3 Amdahl vs Gustafson, (bottom-left) Exercise 4 Amdahl vs Gustafson, (bottom-right) Comparison of Exercise 3 and 4.

Conclusion:

- Exercise 3: Large sequential fraction (30%) limits speedup to 3.3x

- Exercise 4: Tiny sequential fraction allows near-linear speedup
- Matrix multiplication ($O(N^3)$) scales much better than vector operations ($O(N)$)
- Higher computational intensity = better parallel efficiency